

Test Cases — main.dart Bugfixes

Testdokumentation für Flutter App (main.dart)

Datum: 02.06.2026

Tester: Jonathan Schmidt

Version: 1.0

Testumgebung: Flutter Web & Android

TC-001: Driving Mode — Stop-Befehl Timing

Priorität: Kritisch

Kategorie: Funktional

Komponente: Driving Mode

Beschreibung

Test der Stop-Befehl-Verzögerung beim Loslassen von Steuertasten.

Vorbedingungen

- App gestartet
- Driving Mode aktiv
- WebSocket-Verbindung zum Auto hergestellt

Testschritte

1. Taste "W" (Vorwärts) drücken und 2 Sekunden halten
2. Taste loslassen
3. Zeit bis zum Stop-Befehl messen
4. Vorgang mit "A" (Links) wiederholen

Erwartetes Ergebnis

- Auto fährt während Taste gehalten wird
- Stop-Befehl wird **500ms** nach Loslassen gesendet
- Auto stoppt nicht sofort beim Loslassen

Tatsächliches Ergebnis (vor Fix)

FEHLGESCHLAGEN - Stop-Befehl wurde nach nur **10ms** gesendet - Auto stoppte unmittelbar nach jedem Tastendruck - Kontinuierliches Fahren nicht möglich

Tatsächliches Ergebnis (nach Fix)

BESTANDEN - Stop-Befehl wird nach 500ms gesendet - Guard-Bedingung prüft ob Taste noch gehalten wird - Kontinuierliches Fahren funktioniert einwandfrei

Root Cause

```
// Bug: Timer zu kurz, keine Guard-Bedingung
_commandTimer = Timer(const Duration(milliseconds: 10), () {
  activeCommands.clear();
  updateCommand();
});
```

Fix

```
// Fix: 500ms Timer mit Guard
_commandTimer = Timer(const Duration(milliseconds: 500), () {
  if (!keyPressed.values.any((pressed) => pressed)) {
    if (activeCommands.isNotEmpty) {
      activeCommands.clear();
      updateCommand();
    }
  }
});
```

TC-002: Driving Mode — Stop-Befehl beim Button-Release

Priorität: Kritisch

Kategorie: Funktional

Komponente: Driving Mode UI Buttons

Beschreibung

Test ob Stop-Befehle beim Loslassen von UI-Buttons korrekt gesendet werden.

Vorbedingungen

- App gestartet
- Driving Mode aktiv
- Touch-Steuerung verfügbar

Testschritte

1. Gas-Button antippen und halten (2 Sekunden)
2. Button loslassen
3. Beobachten ob Auto stoppt
4. Mit Lenk-Buttons (Links/Rechts) wiederholen
5. Mit Brems-Button wiederholen

Erwartetes Ergebnis

- Auto reagiert während Button gedrückt
- Stop-Befehl wird beim Loslassen gesendet
- Auto stoppt nach Release

Tatsächliches Ergebnis (vor Fix)

FEHLGESCHLAGEN - activeCommands wurde aktualisiert - updateCommand() wurde **nicht** aufgerufen - Kein Stop-Befehl ans Auto gesendet - Auto fuhr weiter

Tatsächliches Ergebnis (nach Fix)

BESTANDEN - updateCommand() wird in allen Release-Pfaden aufgerufen - Stop-Befehl wird zuverlässig gesendet - Auto stoppt korrekt

Betroffene Funktionen

- _pressAccelerate()
- _pressBrake()
- _pressSteerLeft()
- _pressSteerRight()

Fix

```
// Vorher: updateCommand() fehlte
setState(() {
  accelerating = false;
});

// Nachher: updateCommand() hinzugefügt
setState(() {
  accelerating = false;
});
updateCommand(); // *NEU* NEU
```

TC-003: Driving Mode — Watchdog nur für Tastatur

Priorität: Mittel

Kategorie: Funktional

Komponente: Keyboard Handler

Beschreibung

Test dass Watchdog-Timer nur bei Tastatur-Eingaben aktiviert wird, nicht bei Touch-Buttons.

Vorbedingungen

- App im Browser geöffnet
- Driving Mode aktiv

Testschritte

1. Taste “W” drücken und halten
2. Browser-Tab wechseln (Fokus verlieren)
3. Zurück zum App-Tab wechseln
4. Prüfen ob Watchdog Stop-Befehl sendet
5. Gas-Button drücken und halten
6. Prüfen ob Watchdog aktiviert wird

Erwartetes Ergebnis

- Watchdog aktiviert sich bei Tastatur-Eingaben
- Watchdog aktiviert sich **nicht** bei Touch-Buttons
- Stop-Befehl nach 500ms wenn kein Key mehr gehalten

Tatsächliches Ergebnis

BESTANDEN - `_startCommandTimer()` nur in `_handleDrivingKey()` aufgerufen - Touch-Buttons verwenden eigene Release-Handler - Sicherheitsnetz für Browser KeyUp-Event-Verlust funktioniert

Implementierung

```
void _handleDrivingKey(RawKeyEvent event) {
  // ... Key-Handling ...
  if (isDown) {
    _startCommandTimer(); // *NEU* Nur hier, nicht bei Buttons
  }
}
```

TC-004: Driving Mode — Button Long-Press Handling

Priorität: Hoch

Kategorie: UI/UX

Komponente: Control Buttons

Beschreibung

Test der Button-Reaktion bei langem Halten (Long-Press).

Vorbedingungen

- App gestartet
- Driving Mode aktiv

Testschritte

1. Gas-Button antippen und 5 Sekunden halten
2. Button loslassen
3. Prüfen ob `onPointerUp` Event gefeuert wurde
4. Mit allen Steuer-Buttons wiederholen

Erwartetes Ergebnis

- Button reagiert auf Press
- Button reagiert auf Release (auch nach Long-Press)
- Stop-Befehl wird immer gesendet

Tatsächliches Ergebnis (vor Fix)

FEHLGESCHLAGEN - `GestureDetector.onTapUp` wurde bei Long-Press unterdrückt - Flutter klassifizierte Geste als Long-Press - Button blieb "gedrückt" - Stop-Befehl kam nie

Tatsächliches Ergebnis (nach Fix)

BESTANDEN - `Listener.onPointerUp` feuert bedingungslos - Unabhängig von Gesten-Klassifizierung - Stop-Befehl wird immer gesendet

Fix

```
// Vorher: GestureDetector
GestureDetector(
  behavior: HitTestBehavior.opaque,
  onTapDown: (_) => onHold(true),
  onTapUp: (_) => onHold(false),    // *NEU* Wird bei Long-Press unterdrückt
  onTapCancel: () => onHold(false),
)

// Nachher: Listener
Listener(
  behavior: HitTestBehavior.opaque,
  onPointerDown: (_) => onHold(true),
  onPointerUp: (_) => onHold(false),    // *NEU* Feuert immer
  onPointerCancel: (_) => onHold(false),
)
```

TC-005: Draw Route — Kurven-Klassifizierung

Priorität: Hoch

Kategorie: Algorithmus

Komponente: Route Processor

Beschreibung

Test der korrekten Erkennung von Links-/Rechtskurven und geraden Segmenten.

Vorbedingungen

- App gestartet
- Drawing Mode aktiv
- Zeichenfläche bereit

Testschritte

1. Gerade Linie zeichnen (horizontal, 100px)
2. 90° Rechtskurve zeichnen
3. 90° Linkskurve zeichnen
4. Leichte Rechtskurve zeichnen (15°)
5. Playback starten und Befehle beobachten

Erwartetes Ergebnis

- Gerade: `SegmentType.straight`
- 90° rechts: `SegmentType.sharpRight`
- 90° links: `SegmentType.sharpLeft`
- 15° rechts: `SegmentType.rightCurve`

Tatsächliches Ergebnis (vor Fix)

FEHLGESCHLAGEN - Vorzeichen-Konvention invertiert - Links/Rechts vertauscht - Gerade-Schwelle zu groß (10°) - Viele Kurven als "gerade" klassifiziert

Tatsächliches Ergebnis (nach Fix)

BESTANDEN - Positiver Winkel = Rechtskurve - Negativer Winkel = Linkskurve - Gerade-Schwelle: $< 3^\circ$ - Sharp-Schwelle: $> 20^\circ$

Schwellwerte

Parameter	Vorher	Nachher
Gerade	$< 10^\circ$	$< 3^\circ$
Sharp	$> 25^\circ$	$> 20^\circ$
Vorzeichen	invertiert	korrekt

TC-006: Draw Route — Command Timing

Priorität: Hoch

Kategorie: Performance

Komponente: Command Generator

Beschreibung

Test der Command-Dauer-Berechnung für verschiedene Segment-Typen.

Vorbedingungen

- Route gezeichnet mit verschiedenen Segment-Typen
- Playback bereit

Testschritte

1. Route mit 30px geradem Segment zeichnen
2. Route mit 30px Kurve zeichnen
3. Route mit 30px scharfer Kurve zeichnen
4. Playback starten
5. Command-Dauern messen

Erwartetes Ergebnis

- Minimum-Dauer: 80ms
- Kurven-Split: 90% Lenken / 10% Fahren
- Sharp-Multiplikator: 5×

Tatsächliches Ergebnis (vor Fix)

FEHLGESCHLAGEN - Minimum-Dauer: 100ms (zu lang) - Kurven-Split: 70% / 30% (zu wenig Lenkzeit) - Sharp-Multiplikator: 3× (zu wenig für 90° Kurven) - Auto überfuhr Kurven

Tatsächliches Ergebnis (nach Fix)

BESTANDEN - Minimum-Dauer: 80ms - Kurven-Split: 90% / 10% - Sharp-Multiplikator: 5× - Auto navigiert Kurven präzise

Parameter-Änderungen

Parameter	Vorher	Nachher
Minimum-Dauer	100 ms	80 ms
Kurven-Split	70% / 30%	90% / 10%
Sharp-Multiplikator	3×	5×

TC-007: Draw Route — Segment-Mindestabstand

Priorität: Kritisch

Kategorie: Performance

Komponente: Route Processor

Beschreibung

Test der Segment-Generierung und Anzahl bei verschiedenen Mindestabständen.

Vorbedingungen

- Drawing Mode aktiv
- Leere Zeichenfläche

Testschritte

1. Komplexe Route zeichnen (ca. 500px Gesamtlänge)
2. Segment-Anzahl zählen
3. Command-Anzahl zählen

4. Playback-Dauer messen
5. Timer-Drift beobachten

Erwartetes Ergebnis

- 10-15 Segmente pro Route
- ~15-20 Commands
- Playback-Dauer: 5-10 Sekunden
- Minimaler Timer-Drift

Tatsächliches Ergebnis (vor Fix)

FEHLGESCHLAGEN - Mindestabstand: 5px - ~100+ Segmente pro Route - ~120 Befehle generiert - Playback-Dauer: 36+ Sekunden - Akkumulierter Timer-Drift - Auto kam nicht am Ziel an

Tatsächliches Ergebnis (nach Fix)

BESTANDEN - Mindestabstand: 30px - 10-15 Segmente pro Route - ~15-20 Befehle - Playback-Dauer: 5-10 Sekunden - Minimaler Drift

Metriken

Metrik	Vorher (5px)	Nachher (30px)
Segmente	100+	10-15
Commands	~120	~15-20
Dauer	36+ sec	5-10 sec
Drift	Hoch	Minimal

TC-008: Draw Route — Punkt-Glättung

Priorität: Mittel

Kategorie: Algorithmus

Komponente: Route Processor

Beschreibung

Test der Punkt-Glättung für saubere Kurven-Erkennung.

Vorbedingungen

- Drawing Mode aktiv
- Maus/Touch-Eingabe verfügbar

Testschritte

1. Schnell eine wellige Linie zeichnen (Hand-Zittern simulieren)
2. Segment-Klassifizierung beobachten
3. Anzahl der Kurven-Segmente zählen
4. Playback starten und Fahrt beobachten

Erwartetes Ergebnis

- Glatte Kurven trotz zittriger Eingabe
- Wenige, saubere Segmente
- Flüssige Fahrt

Tatsächliches Ergebnis (vor Fix)

FEHLGESCHLAGEN - 3-Punkt-Glättung unzureichend - Viele kleine Zick-Zack-Segmente - Ruckelige Fahrt - Kurven-Erkennung ungenau

Tatsächliches Ergebnis (nach Fix)

BESTANDEN - 5-Punkt gleitendes Mittel - Glatte Segmente - Flüssige Fahrt - Präzise Kurven-Erkennung

Algorithmus

```
// Vorher: 3-Punkt-Glättung
smoothed[i] = (points[i-1] + points[i] + points[i+1]) / 3;

// Nachher: 5-Punkt-Glättung
smoothed[i] = (points[i-2] + points[i-1] + points[i] +
               points[i+1] + points[i+2]) / 5;
```

TC-009: Draw Route — Scale-Faktor für Vollgas-Motor

Priorität: Hoch

Kategorie: Kalibrierung

Komponente: Command Generator

Beschreibung

Test der Distanz-zu-Zeit-Konvertierung für Vollgas-Motor.

Vorbedingungen

- Auto mit Vollgas-Motor (On/Off, keine Geschwindigkeitsregelung)
- Route gezeichnet

Testschritte

1. 30px gerades Segment zeichnen
2. Speed-Multiplier auf 1.0× setzen
3. Erwartete Dauer berechnen
4. Playback starten
5. Tatsächliche zurückgelegte Strecke messen
6. Mit 0.5× wiederholen

Erwartetes Ergebnis

- 30px bei 1.0×: ~120ms
- 30px bei 0.5×: ~240ms
- Auto fährt korrekte Distanz

Tatsächliches Ergebnis (vor Fix)

FEHLGESCHLAGEN - Scale: × 10 - Minimum: 150ms - 30px bei 1.0×: 300ms (zu lang) - Auto überfuhr kurze Strecken deutlich

Tatsächliches Ergebnis (nach Fix)

BESTANDEN - Scale: × 4 - Minimum: 80ms - 30px bei 1.0×: 120ms - Präzise Distanz-Kontrolle

Kalibrierung

Setting	Segment (30px)	Vorher	Nachher
1.0×	Gerade	300 ms	120 ms
0.5×	Gerade	600 ms	240 ms

TC-010: Draw Route — Kurven-Segment-Merge

Priorität: Hoch

Kategorie: Algorithmus

Komponente: Command Generator

Beschreibung

Test des Zusammenführens aufeinanderfolgender Kurven-Segmente.

Vorbedingungen

- Drawing Mode aktiv
- Route mit 90° Ecke

Testschritte

1. Route mit scharfer 90° Rechtskurve zeichnen
2. Segment-Analyse beobachten
3. Anzahl der Kurven-Segmente zählen
4. Anzahl der Lenk-Befehle zählen
5. Playback starten
6. Lenkverhalten beobachten

Erwartetes Ergebnis

- Eine 90° Ecke = 1 Kurven-Segment
- 1 Lenk-Befehl pro Ecke
- Auto lenkt einmal und fährt weiter

Tatsächliches Ergebnis (vor Fix)

FEHLGESCHLAGEN - Eine 90° Ecke *DANN* 2-3 Kurven-Segmente (nach Glättung) - 2-3 separate Lenk-Befehle
- Auto überdrehte - Zu starke Lenkung

Tatsächliches Ergebnis (nach Fix)

BESTANDEN - `_mergeConsecutiveCurves()` implementiert - Aufeinanderfolgende gleich-gerichtete Kurven zusammengefasst - 1 Segment pro Ecke - 1 Lenk-Befehl pro Ecke - Präzise Lenkung

Algorithmus

```
List<RouteSegment> _mergeConsecutiveCurves(List<RouteSegment> segments) {  
    // Fasst aufeinanderfolgende Kurven gleicher Richtung zusammen  
    // leftCurve + leftCurve *DANN* 1× leftCurve  
    // rightCurve + rightCurve *DANN* 1× rightCurve  
}
```

TC-011: Draw Route — State Cleanup

Priorität: Mittel

Kategorie: State Management

Komponente: Playback Controller

Beschreibung

Test der Zustandsbereinigung beim Wechsel von Driving zu Drawing Mode.

Vorbedingungen

- Driving Mode aktiv
- Mehrere Tasten/Buttons gedrückt

Testschritte

1. Im Driving Mode: “W” + “A” gleichzeitig drücken und halten
2. Zu Drawing Mode wechseln
3. Route zeichnen
4. Playback starten
5. Gesendete Befehle beobachten

Erwartetes Ergebnis

- Nur Route-Befehle werden gesendet
- Keine Driving-Mode-Befehle
- Sauberer State

Tatsächliches Ergebnis (vor Fix)

FEHLGESCHLAGEN - `activeCommands` enthielt: `forward`, `left` - `keyPressed` enthielt: W, A - Playback sendete kombinierte Befehle: `forward,left,right` - Auto fuhr unkontrolliert

Tatsächliches Ergebnis (nach Fix)

BESTANDEN - `activeCommands.clear()` am Anfang von `_startPlayback()` - `keyPressed.clear()` am Anfang von `_startPlayback()` - Nur Route-Befehle werden gesendet - Saubere Fahrt

Fix

```
void _startPlayback() async {
  activeCommands.clear(); // *NEU* NEU
  keyPressed.clear();     // *NEU* NEU

  // ... Rest der Playback-Logik
}
```

TC-012: Draw Route — UI Cleanup

Priorität: Niedrig

Kategorie: UI/UX

Komponente: Drawing Page

Beschreibung

Test der UI-Bereinigung (entfernte nicht-funktionale Elemente).

Vorbedingungen

- Drawing Mode geöffnet

Testschritte

1. UI inspizieren
2. Nach Geschwindigkeitsregler suchen
3. Nach Save-Button suchen
4. Code-Review: `_speedMultiplier` Deklaration prüfen

Erwartetes Ergebnis

- Kein Geschwindigkeitsregler sichtbar
- Kein Save-Button sichtbar
- `_speedMultiplier` als `final` deklariert (Wert: 1.0)

Tatsächliches Ergebnis

BESTANDEN - Geschwindigkeitsregler entfernt (macht bei Vollgas-Motor keinen Sinn) - Save-Button entfernt (keine Speicherfunktion vorhanden) - `final double _speedMultiplier = 1.0;` - Saubere, fokussierte UI

Test-Zusammenfassung

Statistik

- **Gesamt Tests:** 12
- **Kritisch:** 3
- **Hoch:** 5
- **Mittel:** 3
- **Niedrig:** 1

Ergebnisse (vor Fixes)

- *FEHLGESCHLAGEN* **Fehlgeschlagen:** 11
- *BESTANDEN* **Bestanden:** 1

Ergebnisse (nach Fixes)

- *BESTANDEN* **Bestanden:** 12
- *FEHLGESCHLAGEN* **Fehlgeschlagen:** 0

Kategorien

- **Funktional:** 6 Tests
- **UI/UX:** 3 Tests
- **Algorithmus:** 4 Tests
- **Performance:** 2 Tests
- **State Management:** 1 Test

Kritische Bugs behoben

1. Stop-Befehl Timing (TC-001)
 2. Fehlende `updateCommand()` Aufrufe (TC-002)
 3. Segment-Mindestabstand Performance (TC-007)
-

Lessons Learned

1. **Timer-Werte kritisch:** 10ms vs 500ms macht den Unterschied zwischen funktionierend und nicht-funktionierend
2. **Flutter Gesture System: GestureDetector** unterdrückt Events bei Long-Press *DANN Listener* verwenden
3. **State Management:** Globale States müssen beim Mode-Wechsel bereinigt werden
4. **Algorithmus-Kalibrierung:** Vollgas-Motor benötigt andere Parameter als Geschwindigkeits-geregelte Motoren
5. **Glättung wichtig:** 5-Punkt-Glättung deutlich besser als 3-Punkt für Kurven-Erkennung

Dokumentiert von: Alexa van der Meulen

Review: Team R.E.D.

Status: Alle Tests bestanden *BESTANDEN*